Exception Handling and Error Management

Robust stream pipelines must handle exceptions gracefully to ensure reliability, data integrity, and uninterrupted processing.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

Exception Handling and Error Management in Java

Handle Exceptions Gracefully, Keep Applications Reliable

1. WHAT IS AN EXCEPTION?

An exception is an event that occurs during program execution that disrupts the normal flow of instructions.

Examples:

- Division by zero
- Null pointer reference
- Invalid input
- File not found

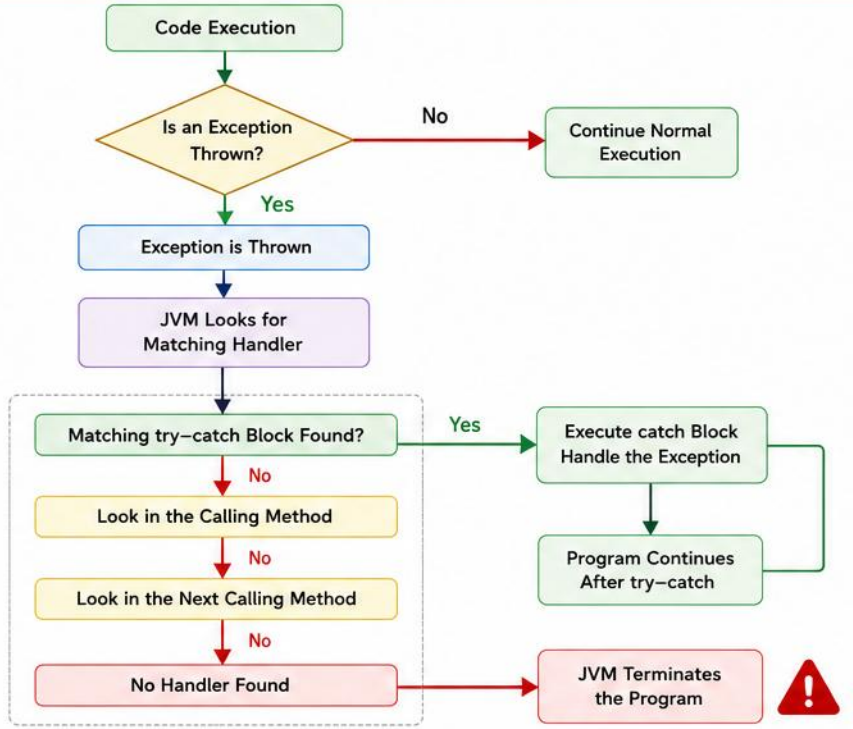
2. TYPES OF EXCEPTIONS

Checked Exceptions
Checked at compile time.
Must be handled or declared.
Example: IOException, SQLException

Unchecked Exceptions
Checked at runtime.
Not mandatory to handle.
Example: NullPointerException, ArithmeticException

Errors
Serious problems beyond application control.
Example: OutOfMemoryError, StackOverflowError

3. HOW EXCEPTION HANDLING WORKS



4. KEYWORDS USED

try	Encloses code that might throw an exception.
catch	Catches and handles the exception.
finally	Executes always, whether exception occurs or not.
throw	Explicitly throws an exception.
throws	Declares exceptions that a method might throw.

5. BEST PRACTICES

- Handle exceptions at the appropriate level.
- Provide meaningful error messages.
- Always clean up resources in finally block or try-with-resources.
- Avoid catching generic Exception.
- Log exceptions for debugging and monitoring.

6. EXAMPLE

```
try {
     int result = 10 / 0;   // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
    System.out.println("Execution completed");
}
```

Output:

```
Cannot divide by zero: / by zero
Execution completed
```

Explanation:

The exception is caught and handled in the catch block.
The finally block executes regardless.

Remember

Exception handling improves code reliability, user experience, and helps in debugging and maintenance.

Good code anticipates problems and handles them gracefully.

MODULE 7 OVERVIEW

Robust stream pipelines must handle exceptions gracefully to ensure reliability, data integrity, and uninterrupted processing.

LEARNING OBJECTIVES

1. Understand exceptions in stream pipelines; handle exceptions within lambda expressions.
2. Use try-catch, Optional, and fallback strategies.
3. Log and propagate errors effectively; ensure data quality and pipeline resilience.

WHY EXCEPTION HANDLING MATTERS

- Prevents pipeline termination due to bad data; ensures partial success and maximum data throughput.
- Improves reliability and maintainability; provides meaningful error information for debugging.
- Protects downstream systems from invalid data.
- Shows up everywhere: parsing (NumberFormatException), database access (SQLException), file I/O (IOException), and business validation — handle each close to its source, never with an empty catch block.

KEY TAKEAWAY Streams stop on unhandled exceptions. Handle, log, or propagate — don't ignore. Resilience is key to production pipelines.

ERROR HANDLING IN JAVA

Handle exceptions gracefully to build robust and reliable applications

ERRORS vs EXCEPTIONS

Errors

- Serious problems that a program should not try to catch.
- Usually related to system failures.
- Examples: `OutOfMemoryError`, `StackOverflowError`

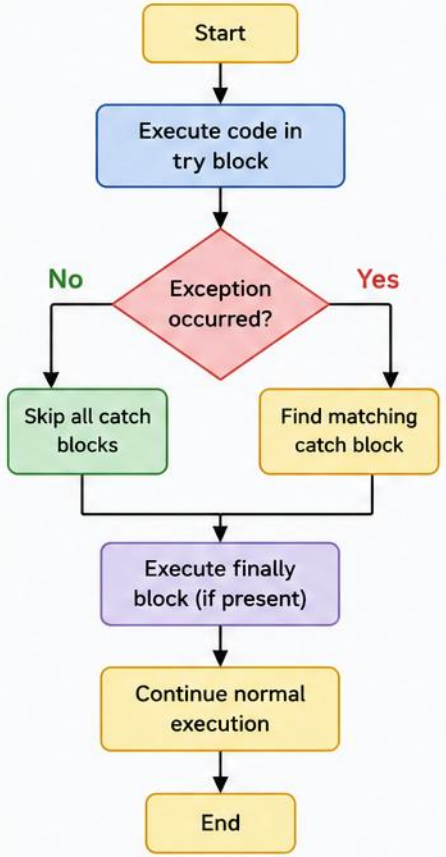
Exceptions

- Conditions that a program can catch and handle.
- Can be checked (compile-time) or unchecked (runtime).
- Examples: `IOException`, `ArithmeticException`

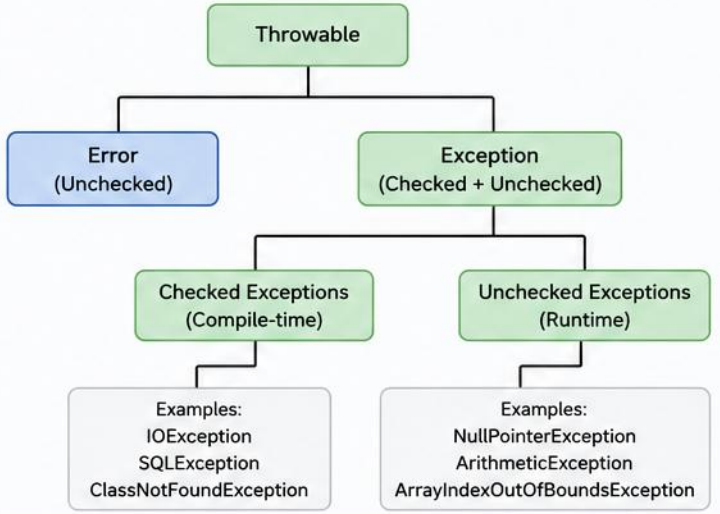
KEY POINTS

1. Use `try` to enclose risky code.
2. Use `catch` to handle specific exceptions.
3. Use `finally` to execute code that must run regardless of exception.
4. You can have multiple `catch` blocks.
5. Throw meaningful exceptions using `throw` keyword.

TRY–CATCH–FINALLY FLOW



EXCEPTION HIERARCHY



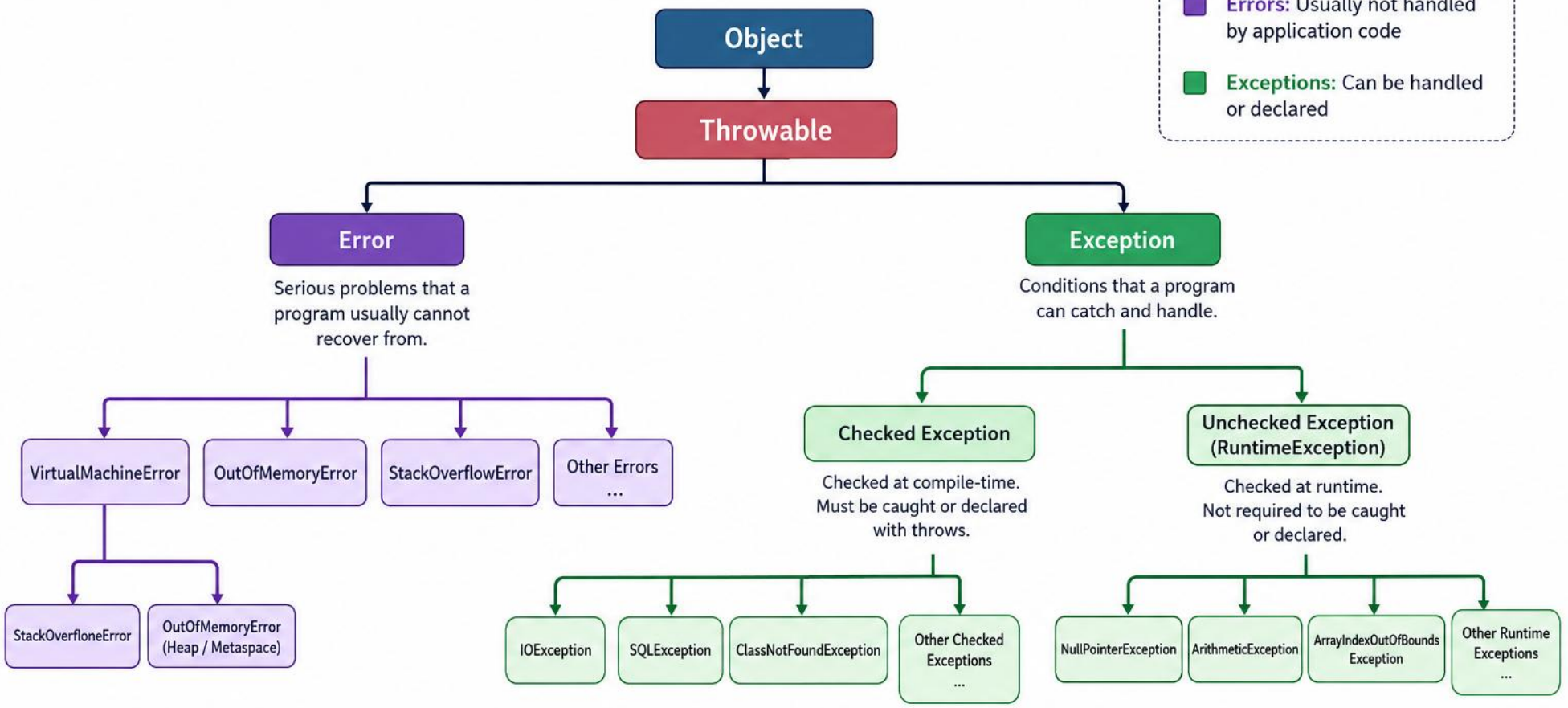
EXCAMPLE

```
public class Example {
   public static void main(String[] args) {
      try {
         int result = 10 / 0;  // risky code
      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero: " + e.getMessage());
      } finally {
         System.out.println("This will always execute.");
      }
   }
}
```

Good error handling improves user experience, helps in debugging, and prevents unexpected program crashes.

© Dr. Gurinderjeet Kaur

Java Exception Hierarchy



- **Throwable** is the root of the exception hierarchy.
- **Error** represents serious issues that are beyond the control of applications.
- **Exception** represents conditions that applications can handle.

✓ **Checked Exceptions** must be caught or declared.

✓ **Unchecked Exceptions** (RuntimeException and its subclasses) are not required to be caught or declared.

Best Practice: Handle exceptions gracefully to build robust and reliable applications.

STREAM NOTE Checked exceptions must be handled inside lambdas (catch or wrap); an unhandled unchecked exception can still terminate a stream pipeline.

CHECKED EXCEPTIONS IN JAVA

Checked exceptions are verified at compile time.
The compiler enforces that they must be either caught or declared.

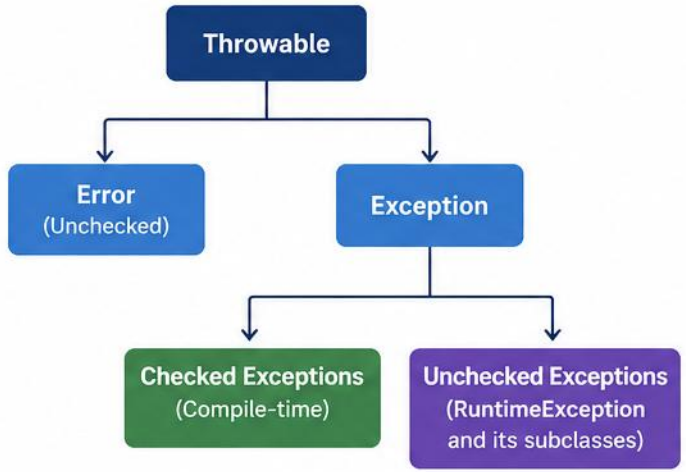
WHAT ARE CHECKED EXCEPTIONS?

Exceptions that are checked by the compiler at compile time.

They are subclasses of Exception (excluding RuntimeException).

KEY CHARACTERISTICS

- Checked at compile time.
- Must be either caught using try–catch or declared using throws.
- They represent conditions that a well-written program should handle.
- They are part of the Exception hierarchy (excluding RuntimeException).



COMMON EXAMPLES

- **IOException** (when reading/writing files)
- **SQLException** (database access errors)
- **URISyntaxException** (invalid URI syntax)
- **ClaseNotFoundException** (class not found at runtime)

HOW TO HANDLE CHECKED EXCEPTIONS

1. HANDLE USING try–catch

Handle the exception inside the method using try–catch.

```
import java.io.*;
public class Example {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("data.txt");
            int ch = fr.read();
            System.out.println((char) ch);
            fr.close();
        } catch (IOException e) {
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
```

2. DECLARE USING throws

Declare the exception using throws and let the caller handle it.

```
import java.io.*;

public class Example {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("data.txt");
        int ch = fr.read();
        System.out.println((char) ch);
        fr.close();
    }
}
```

BEST PRACTICE Handle checked exceptions meaningfully and provide clear messages or recovery options. | Do not ignore or swallow exceptions.

CHECKED EXCEPTIONS — EXAMPLE & HANDLING OPTIONS

Reading a file with FileReader — if IOException isn't caught or declared, the code won't compile.

```
// 1. Handle using try-catch
try {
    FileReader fr = new FileReader("a.txt");
} catch (IOException e) {
    // handle it
}
```

```
// 2. Declare using throws
public void readFile()
        throws IOException {
    // risky code
}
```

- Common checked-exception sources: File I/O, network operations, thread coordination, database access, reflection & class loading, external systems.

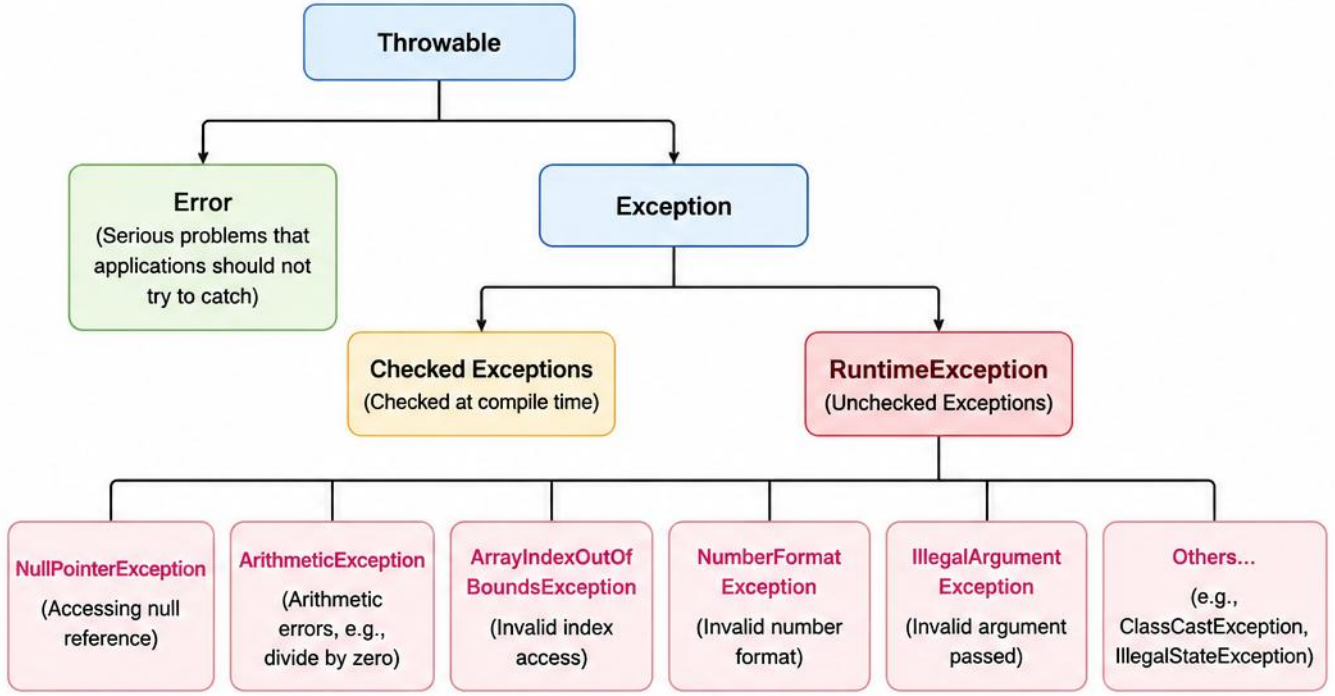
KEY TAKEAWAY Checked exceptions help you build reliable pipelines by ensuring anticipated problems are handled deliberately, not left to chance.

Unchecked Exceptions in Java

Runtime exceptions that occur due to programming mistakes.

Key Points

- Unchecked exceptions are subclasses of **RuntimeException**.
- They occur at **runtime**.
- Mostly caused by **programming** errors (logic bugs).
- Not checked at compile time. No need to catch or declare using **throws**.
- If not handled, the program terminates abnormally.



Example

```java
public class UncheckedExample {
    public static void main(String[] args) {
        String s = null;
        System.out.println(s.length()); // This will throw NullPointerException
        int result = 10 / 0;
        System.out.println(result);     // This will throw ArithmeticException
    }
}
```

How to Handle?

- Use try-catch (recommended for graceful handling).
- Validate inputs and add proper checks in code.
- Fix the root cause (these are usually coding mistakes).

Unchecked exceptions are not the compiler's responsibility.
They are the **programmer's responsibility!**

UNCHECKED EXCEPTIONS — HANDLING & WHEN THEY OCCUR

Catch defensively, or better yet, prevent them before they happen.

```java
// 1. Catch (only at a recovery boundary)
try {
    process(obj);
} catch (NullPointerException e) {
    // handle it
}
```

```java
// 2. Validate & prevent
if (obj != null) {
    obj.doWork();
}
```

- Common causes: programming bugs, bad assumptions, edge cases, invalid data/state, external inputs, type mismatch.

KEY TAKEAWAY Catch unchecked exceptions only at a meaningful recovery boundary. Prefer preventing NullPointerException, IllegalArgumentException, and index errors through validation and correct logic.

PRE-LAB EXERCISES OVERVIEW

Eight short warm-ups to complete before starting the official Lab 7 (ATM Banking System).

KEY CONCEPTS COVERED

- Exception Hierarchy · Try-Catch Blocks · Finally & Try-With-Resources · Throw & Throws · Custom Exceptions · Propagation · Error Handling Strategies · Logging & Diagnostics · Best Practices.

PRE-LAB EXERCISES (1–8)

1. Trigger common exceptions — ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException.
2. try-catch-finally — simulate a transfer; finally always prints “cleanup”.
3. try-with-resources — read a small file; the resource closes automatically.
4. throw and throws — validateAmount() rejects amounts ≤ 0.
5. Custom exception — checked InsufficientFundsException thrown from withdraw().
6. Exception propagation — A→B→C call chain; catch only at the top.
7. Error handling strategies — retry a flaky fetchBalance() call, then fall back to a safe default.
8. Logging warm-up — log an exception with account-id context.

Next: open Lab 7 — the official ATM Banking System lab.

KEY TAKEAWAY Environment: JDK 21, IntelliJ IDEA Community (or VS Code), plain console output — no external logging framework required.

TRY IT YOURSELF — EXERCISE 1: TRIGGER COMMON EXCEPTIONS

Reinforces: unchecked exceptions, as just covered.

Goal

- Trigger three common runtime exceptions in isolated try blocks, catch each specifically

ArithmeticException

- int result = 10 / 0; — division by zero

NullPointerException

- String value = null; value.length(); — calling a method on null

ArrayIndexOutOfBoundsException

- values[5] on a 2-element array — index out of range

Prove

- main continues after each catch — the program doesn't crash

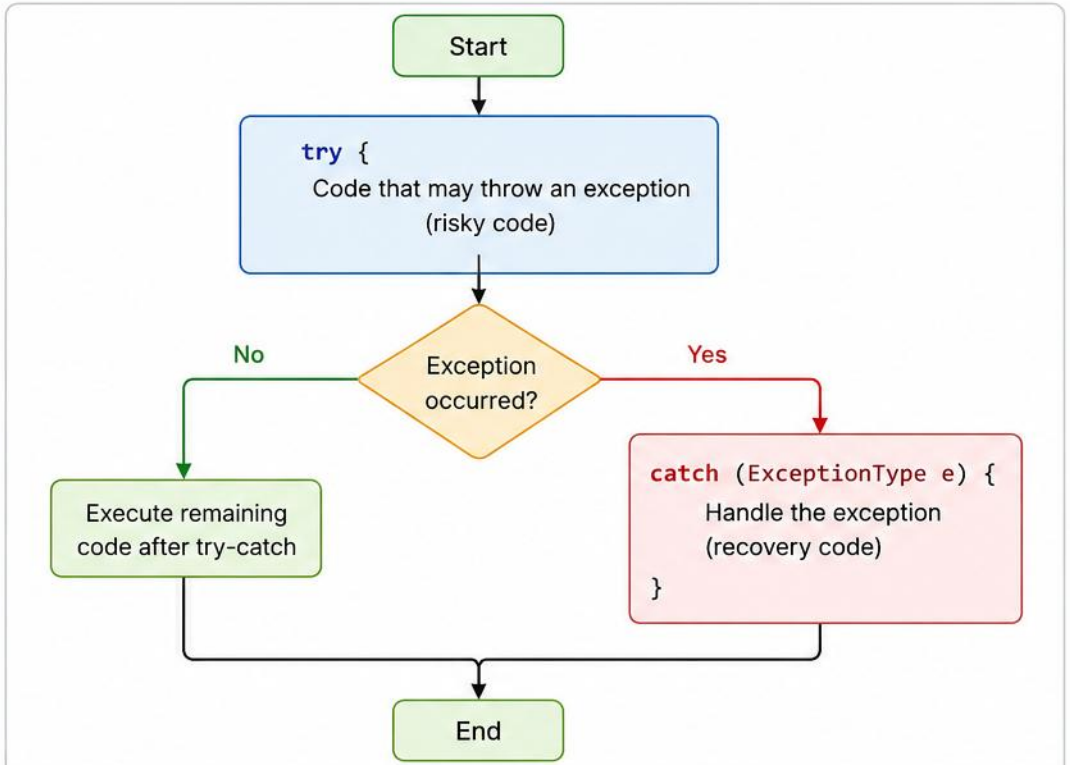
Reference

- exercises/exercise-01-common-exceptions.md

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-common-exceptions.md.

Try-Catch Blocks in Java

Handling Exceptions Gracefully



Example

```
try {
    int result = 10 / 0;  // Risky code
    System.out.println(result);
} catch (AritmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
}
System.out.println("Program continues...");
```

Try-Catch Structure

```
try {
    // Code that may throw an exception
}
catch (ExceptionType e) {
    // Code to handle the exception
}
```

How it Works

- The `try` block contains code that might throw an exception.
- If no exception occurs, the program continues after the `try-catch` block.
- If an exception occurs, the control is immediately transferred to the `catch` block.
- The `catch` block handles the exception and the program continues execution after it.

Key Benefits

- Prevents abnormal program termination
- Allows graceful error handling and recovery
- Improves code robustness and user experience

© Dr. Gurinderjeet Kaur

TRY-CATCH — WORKED EXAMPLE

Multiple catch blocks, ordered from specific to general.

```
Scanner sc = new Scanner(System.in);
try {
    System.out.print("Enter a number: ");
    int num = sc.nextInt();
    int result = 100 / num;
    System.out.println("Result = " + result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Invalid input: " + e.getMessage());
} finally {
    sc.close();
}
// Rule: order specific → general; log with context, not printStackTrace()
```

KEY TAKEAWAY Specific exceptions must be caught before general ones — if a parent exception is caught first, child exceptions become unreachable.

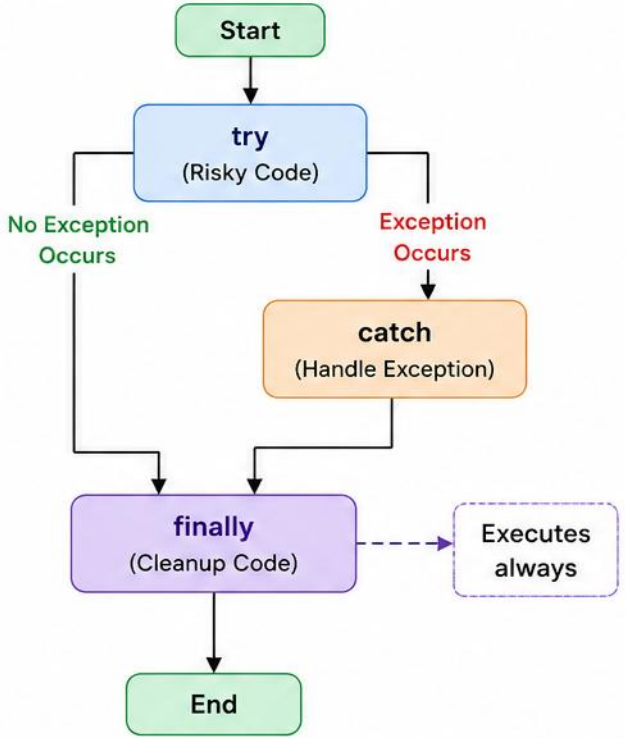
finally Block in Java

The **finally** block always executes after the **try** and **catch** blocks, regardless of whether an exception occurs or not.

Purpose of finally Block

- Used to close resources (connections, streams, files, etc.)
- Executes cleanup code
- Helps in releasing resources even if an exception occurs
- Ensures important code runs in all scenarios

Execution Flow



Key Points

- **finally** block is executed whether an exception is handled or not.
- It is executed even if **return** statement is present in **try** or catch.
- It runs even if **System.exit()** is called.
- Used mainly for cleanup activities.

Example

```
public class FinallyExample {
   public static void main(String[] args) {
      try {
          int data = 10 / 0;  // Exception
      } catch (ArithmeticException e) {
          System.out.println("Exception: " + e.getMessage());
      } finally {
          System.out.println("Finally block executed");
      }
      System.out.println("End of program");
   }
}
```

Output (When Exception Occurs)

```
Exception: / by zero
Finally block executed
End of program
```

Remember:
finally block is your safety net –
it guarantees that important code runs, always.

FINALLY BLOCK — EXAMPLE & KEY POINTS

A finally block runs whether the division succeeds or throws.

```
try {
    int result = 10 / 2;
    System.out.println("Result: " + result);
} finally {
    System.out.println("Cleanup done.");
}
// Output: Result: 5  Cleanup done.
```

- finally executes after try or catch; used for cleanup (files, connections, sockets, streams).
- If an exception occurs in finally, it can override a previous exception. Important: finally may not execute if the JVM terminates abruptly — e.g. System.exit() or a crash.

KEY TAKEAWAY If return is used in try/catch, finally still runs before control actually returns.

TRY IT YOURSELF — EXERCISE 2: TRY-CATCH-FINALLY

Reinforces: finally, as just covered.

🎯 Goal

- Compare a successful and a failed transfer; confirm cleanup runs after both

✅ Success path

- transfer(false) — try block completes, no exception

🛑 Failure path

- transfer(true) — throws, caught by catch(IllegalStateException)

🧹 finally

- Runs "Cleanup: release transfer session." on both paths

💡 Key concept

- finally always executes, whether the try succeeds, fails, or is caught

📖 Reference

- exercises/exercise-02-try-catch-finally.md

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-try-catch-finally.md.

try-with-resources in Java

Automatically closes resources that implement **AutoCloseable** (e.g., File, Stream, Connection) at the end of the try block.

SYNTAX

```
try (ResourceType resource = new ResourceType(args)) {
    // use the resource
} catch (ExceptionType e) {
    // handle exception
} finally {
    // optional
}
```

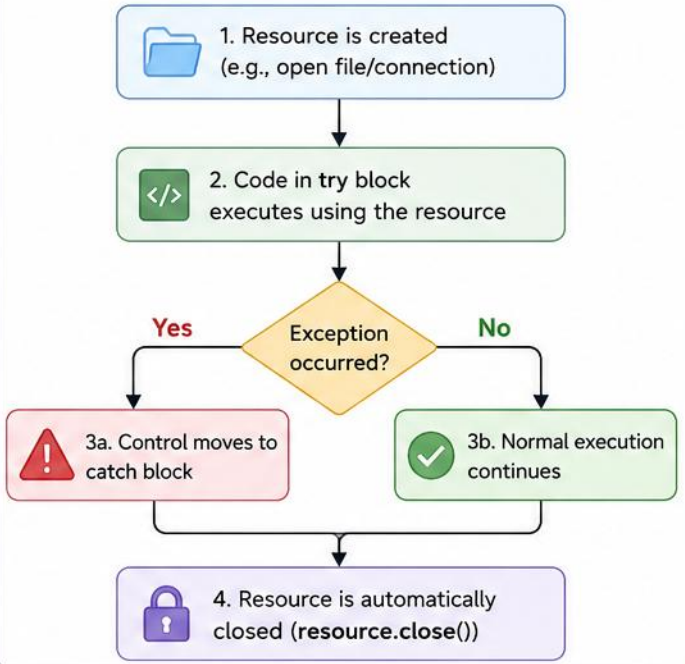
Resource is declared inside the parentheses.

EXAMPLE

```
try (BufferedReader reader = new BufferedReader(
      new FileReader("input.txt"))) {
  String line;
  while ((line = reader.readLine()) != null) {
    System.out.println(line);
  }
} catch (IOException e) {
  System.out.println("Error reading file: " + e.getMessage());
}
```

`reader` is automatically closed at the end of the try block, whether an exception occurs or not.

HOW IT WORKS



BENEFITS

- Ensures resources are closed automatically.
- Prevents resource leaks.
- Cleaner and more concise code.
- Works with any resource that implements **AutoCloseable**.

© Dr. Gurinderjeet Kaur

TRY-WITH-RESOURCES — EXAMPLE

Reading a file — the BufferedReader closes automatically, even if an exception occurs.

```
try (BufferedReader br =
        new BufferedReader(new FileReader("a.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

KEY POINT: if an exception occurs while closing resources, the exception from the try block is preserved, and the closing exception is suppressed.
WHEN TO USE: any Closeable/AutoCloseable resource — files, database connections, sockets, and streams.

KEY TAKEAWAY Multiple resources can be declared in the same try — they close in reverse order automatically.

FINALLY VS. TRY-WITH-RESOURCES — WHEN TO USE EACH

A quick decision guide for choosing between the two cleanup mechanisms.

- AutoCloseable resource (file, socket, DB connection) → use try-with-resources; Java closes it automatically.
- Non-resource cleanup (resetting state, clearing a flag) → use finally; there is nothing here for try-with-resources to close.
- Multiple resources to close together → use try-with-resources; they close automatically in reverse order of declaration.
- Locks, semaphores, or session flags outside AutoCloseable → finally may be the appropriate tool since there is no resource to declare.

KEY TAKEAWAY Rule of thumb: if it implements AutoCloseable, let try-with-resources close it — reach for finally only for cleanup that isn't a resource.

TRY IT YOURSELF — EXERCISE 3: TRY-WITH-RESOURCES

Reinforces: try-with-resources, as just covered.

Goal

- Read a small file with BufferedReader in try-with-resources — no manual close()

Setup

- Files.writeString() creates transactions.txt with two lines

Read

- try (BufferedReader reader = Files.newBufferedReader(file)) { ... }

Auto-close

- reader.close() runs automatically when the try block exits

Key concept

- Any AutoCloseable resource closes itself, even if an exception is thrown

Reference

- exercises/exercise-03-try-with-resources.md

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-try-with-resources.md.

Java throw and throws Keywords

Both are used in exception handling but serve different purposes.

throw

The **throw** keyword is used to **explicitly** throw an exception from a method or a block.

How it works

Normal Execution → `throw new Exception();` → Exception is thrown

Example

```
public void validateAge(int age) {
   if (age < 18) {
       throw new IllegalArgumentException("Age must be 18 or above");
   }
   System.out.println("Age is valid");
}
```

- Creates an exception object and throws it.
- Transfers control to the nearest **catch** block.

throws

The **throws** keyword is used in a method signature to declare that the method may throw one or more exceptions. It **does not throw** the exception; it passes the responsibility to the caller.

How it works

Method Call → Method with throws Exception → Caller handles the Exception (try-catch or further throws)

Example

```
import java.io.*;

public void readFile() throws IOException {
   FileReader fr = new FileReader("file.txt");  // may throw IOException
   BufferedReader br = new BufferedReader(fr);
}
```

- Does not create or throw the exception.
- Declares the exception so the **caller** must handle it.

Key Differences

Aspect	throw	throws
Purpose	Explicitly throws an exception.	Declares that a method may throw an exception.
Where Used	Inside method or block.	In method signature.
Action	Creates and throws an exception object.	Does not throw; passes responsibility to the caller.
Number of Exceptions	Throws one exception at a time.	Can declare multiple exceptions.
Example	`throw new IOException("Error occurred");`	`public void read() throws IOException, FileNotFoundException`

Best Practice

- Use **throw** when you want to raise an exception manually.
- Use **throws** when a method might encounter an exception that it cannot handle and wants the caller to handle it.

In short: **throw** = you throw it now, **throws** = I may throw it, you handle it.

THROW AND THROWS — COMBINED EXAMPLE

A method that declares a checked exception, and explicitly throws it.

```
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException(
            "Insufficient balance for withdrawal");
    }
    balance -= amount;
}
```

- If a method calls another method that throws a checked exception, it must either handle it or declare it using throws.

KEY TAKEAWAY throws is mandatory for unhandled checked exceptions; unchecked exceptions (RuntimeException and subclasses) never need to be declared.

TRY IT YOURSELF — EXERCISE 4: THROW VS THROWS

Reinforces: throw and throws, as just covered.

Goal

- Distinguish actively throwing an exception from declaring one may propagate

throw

- validateAmount(-10) — throw new IllegalArgumentException(...) fires now

throws

- String loadPolicy(Path) throws IOException — declares a checked contract

Difference

- throw raises one exception; throws is a method-signature declaration

Key concept

- throws never fires anything by itself — it only documents what might propagate

Reference

- exercises/exercise-04-throw-throws.md

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-throw-throws.md.

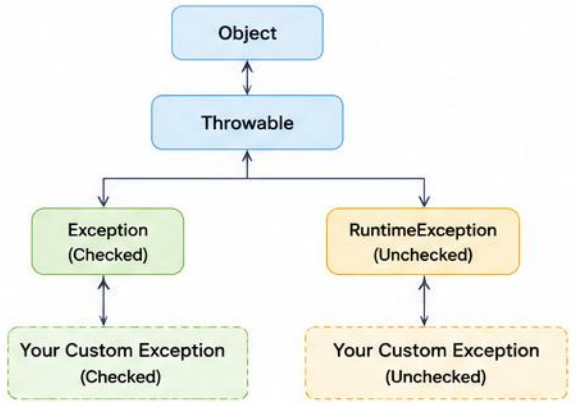
Creating Custom Exceptions in Java

Custom exceptions help create meaningful, domain-specific error handling.

STEPS TO CREATE A CUSTOM EXCEPTION

1. **Extend Exception** — Create a class that extends Exception (or RuntimeException).
2. **Add Constructors** — Provide constructors to pass custom messages and causes.
3. **Throw the Exception** — Use the *throw* keyword to throw the custom exception.
4. **Handle the Exception** — Catch and handle the custom exception in calling code.

EXCEPTION HIERARCHY (Simplified)



EXAMPLE: CUSTOM CHECKED EXCEPTION

```java
// 1. Create Custom Exception
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException() {
        super("Insufficient balance in account.");
    }
    public InsufficientBalanceException(String message) {
        super(message);
    }
    public InsufficientBalanceException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```java
// 2. Throw Custom Exception
public class BankService {
  public void withdraw(double amount, double balance)
        throws InsufficientBalanceException {
    if (amount > balance) {
        throw new InsufficientBalanceException(
            "Cannot withdraw " + amount +
            ". Available balance: " + balance
        );
    }
    // process withdrawal
}
```

3. Handle Custom Exception

```java
public class Main {
  public static void main(String[] args) {
      BankService service = new BankService();

    try {
        service.withdraw (1000, 500);
    } catch (InsufficientBalanceException ex) {
        System.out.println("Error: " + ex.getMessage());
    }
}
```

BEST PRACTICES

- Use meaningful names that describe the error.
- Provide clear and helpful error messages.
- Provide multiple constructors (no-arg, message, cause).
- Extend Exception for checked exceptions (recoverable conditions).
- Extend RuntimeException for unchecked exceptions (programming errors).

WHEN TO USE CUSTOM EXCEPTIONS?

- When built-in exceptions do not clearly describe the problem.
- To represent domain-specific error conditions.
- To improve code readability and error handling.

Examples:

- InvalidAgeException
- InsufficientBalanceException
- OrderNotFoundException
- InvalidLoginException

Key Takeaway: Custom exceptions make your code more expressive, maintainable, and easier to debug by providing meaningful error information.

CUSTOM EXCEPTION — CLASS & USAGE

A checked custom exception that carries a business-relevant field.

```
// Checked Custom Exception
public class InvalidAgeException extends Exception {
    private int age;
    public InvalidAgeException(String message, int age) {
        super(message);
        this.age = age;
    }
    public int getAge() { return age; }
}
```

- Common constructors: default, (String message), (String message, Throwable cause), plus any custom fields you add.

KEY TAKEAWAY Custom exceptions make code more expressive — they communicate the exact problem to the caller.

CUSTOM EXCEPTIONS — WHEN TO USE & CHECKED VS UNCHECKED

Three named examples, and the decision that shapes their design.

EXAMPLES

InsufficientBalanceException

User balance too low.

InvalidOrderException

Order data is invalid.

ResourceNotFoundException

Resource not found.

	CHECKED (extends Exception)	UNCHECKED (extends RuntimeException)
`Use when`	Caller is expected to handle a foreseeable condition	Programming-contract violation; recovery not guaranteed

ACCURACY NOTE Use checked exceptions when callers are expected and required to handle a foreseeable condition. Use unchecked exceptions for programming-contract violations or when forcing catch/declare adds little value. Recovery is not determined solely by whether an exception is checked or unchecked.

KEY TAKEAWAY Choose between checked and unchecked wisely — recovery isn't determined by that choice alone.

TRY IT YOURSELF — EXERCISE 5: CUSTOM CHECKED EXCEPTION

Reinforces: creating custom exceptions, as just covered.

Goal

- Model insufficient balance as a meaningful checked domain exception

File

- InsufficientFundsException.java extends Exception

Context

- Preserves balance and requested amount as structured fields

Message

- "Insufficient funds: balance=%.2f, requested=%.2f".formatted(...)

Key concept

- A checked custom exception forces callers to acknowledge a real domain failure

Reference

- exercises/exercise-05-custom-exception.md

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-custom-exception.md.

EXCEPTION PROPAGATION IN JAVA

When an exception occurs, it is thrown from the current method and propagated up the **call stack** until it is caught.

NORMAL CALL FLOW

```
main()
public static void main(String[] args) {
    methodA();
}
```

```
methodA()
void methodA() {
   methodB();
}
```

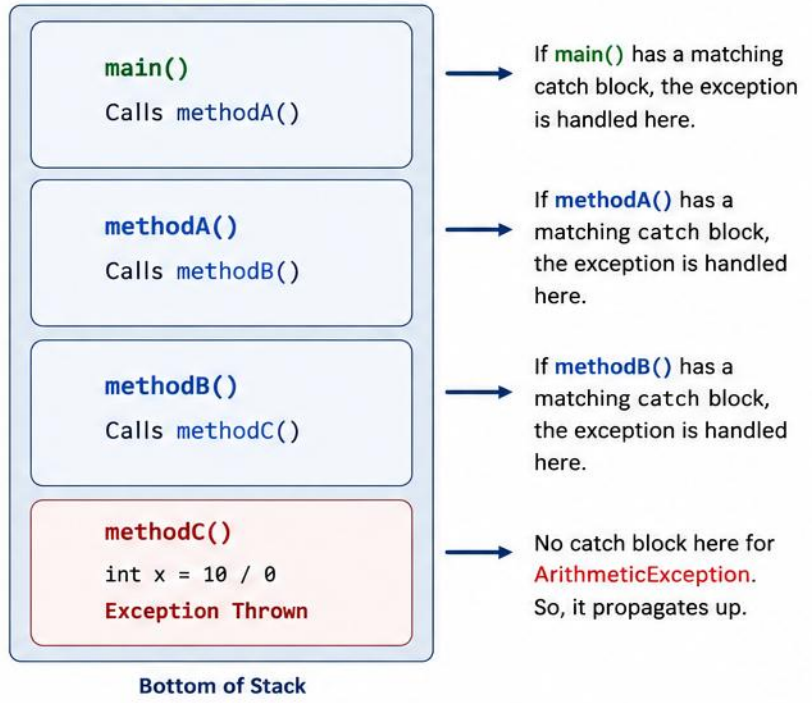
```
methodB()
void methodB() {
   methodC();
}
```

```
methodC()
void methodC() {
   int x = 10 / 0;  // ArithmeticException
    ✕ Exception occurs here!
}
```

EXCEPTION PROPAGATION (UNWINDING THE STACK)

1. Exception thrown in `methodC()`.
2. Not caught in `methodC()`, so it is passed to `methodB()`.
3. Not caught in `methodB()`, so it is passed to `methodA()`.
4. Not caught in `methodA()`, so it is passed to `main()`.

CALL STACK (RUNTIME)



KEY POINTS

- When an exception occurs, Java creates an exception object and looks for a matching catch block.
- If not found in the current method, the exception is passed back to the caller.
- This continues up the call stack until it is caught.
- If no method handles it, the program terminates and the default exception handler prints the stack trace.

EXAMPLE OUTPUT (IF NOT CAUGHT)

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
      at com.example.MyClass.methodC(MyClass.java:15)
      at com.example.MyClass.methodB(MyClass.java:10)
      at com.example.MyClass.methodA(MyClass.java:6)
      at com.example.MyClass.main(MyClass.java:3)
```

PROPAGATION — CODE EXAMPLE & RULES

The classic three-method chain, ending in main().

```
void methodC() { int x = 10 / 0; }        // throws here
void methodB() { methodC(); }             // doesn't handle
void methodA() { methodB(); }             // doesn't handle
public static void main(String[] args) {
    try { new App().methodA(); }
    catch (ArithmeticException e) { /* handled here */ }
}
```

- Propagation moves up the call stack, one method at a time; if a method catches it, propagation stops there.
- Only if a method doesn't catch the exception is it propagated further; if no method catches it, the JVM handles it.

KEY TAKEAWAY Output: Exception handled in main(): java.lang.ArithmeticException: / by zero.

EXCEPTION PROPAGATION — BEST PRACTICES

Seven habits for propagating exceptions cleanly instead of just passing them along.

- Handle exceptions where you can meaningfully recover; don't catch exceptions only to rethrow them without adding value.
- Provide clear error messages; log the exception before rethrowing or exiting.
- Use custom exceptions to add context when propagating.
- Avoid catching Exception or Throwable unless necessary; let unexpected exceptions propagate to a global handler.

KEY TAKEAWAY If you don't handle it, it will keep climbing until someone does!

TRY IT YOURSELF — EXERCISE 6: EXCEPTION PROPAGATION

Reinforces: exception propagation, as just covered.

🎯 Goal

- Trace a checked exception from account layer → service → menu → main

🏦 accountLayer()

- throws InsufficientFundsException — the origin of the failure

⬆️ serviceLayer(), menuLayer()

- Both just declare throws and let it propagate untouched

🎧 Catch point

- Only main catches it — the one true recovery boundary

💡 Key concept

- Catch only where you can meaningfully recover, not at every layer

📖 Reference

- exercises/exercise-06-propagation.md

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-propagation.md.

ERROR HANDLING STRATEGIES

Approaches to detect, manage, and recover from errors so applications stay robust, reliable, and user-friendly.

KEY PRINCIPLES

- Understand what can go wrong; handle appropriately by choosing the right strategy; be consistent — follow standard patterns.
- Don't hide errors — log and escalate when needed; put the user first with meaningful messages.
- Recover or fail — the system should either recover or fail safely.

KEY TAKEAWAY Good error handling is not just about catching errors — it's about building trust, reliability, and a great user experience.

ERROR HANDLING STRATEGIES IN JAVA

Goal: Detect errors, handle them gracefully, and maintain normal application flow.

1. Try–Catch Block

Handle exceptions locally using try–catch to prevent abnormal termination.

```
try {
   // risky code
} catch (Exception e) {
   // handle exception
}
```

Best for:
Recoverable errors that can be handled and app can continue.

2. Multiple Catch

Handle different types of exceptions differently in separate catch blocks.

```
try {
   // code
} catch (IOException e) {
   // handle IO
} catch (NumberFormatException e) {
   // handle number format
```

Best for:
When different exceptions require different handling logic.

3. Finally Block

Execute important cleanup code regardless of whether an exception occurs.

```
try {
   // code
} catch (Exception e) {
   // handle
} finally {
   // cleanup code
}
```

Best for:
Releasing resources (closing files, DB connections, etc.).

4. Throw Keyword

Manually throw an exception when a business rule or condition is violated.

```
if (age < 18) {
   throw new
    IllegalArgumentException
     ("Age must be 18+");
}
```

Best for:
Validating conditions and signaling errors explicitly.

5. Throws Keyword

Declare exceptions in method signature and delegate handling to the caller.

```
void readFile() throws
  IOException {
   // code
}
```

Best for:
Letting the caller handle the exception (exception propagation).

BEST PRACTICES

Catch Specific Exceptions
Avoid catching Exception or Throwable.

Log Exceptions
Always log exceptions with meaningful messages.

Don't Ignore Exceptions
Ignoring exceptions can hide bugs and cause failures.

Provide Helpful Messages
Use clear messages to help in debugging and support.

Keep It Simple & Consistent
Use consistent error handling across your application.

Remember: Good error handling improves application reliability, user experience, and makes debugging easier.

COMMON ERROR HANDLING STRATEGIES

Six patterns cover most real-world error scenarios — pick based on the failure's nature and impact.

Retry

- Retry transient failures (network blips, timeouts) with backoff.

Fallback / Default

- Provide a safe default or cached value when an operation fails.

Skip & Continue

- Isolate a bad record, log it, and keep the pipeline moving.

Fail Fast

- Stop immediately for unrecoverable or unsafe conditions.

Graceful Degradation

- Reduce functionality but keep the system available.

Circuit Breaker

- Stop calling a failing dependency until it recovers.

KEY TAKEAWAY Strategy selection guide: match the strategy to the error type, business impact, and whether the failure is transient or permanent.

ERROR HANDLING STRATEGIES — DECISION GUIDE

Match each failure scenario to the strategy that fits it best.

- Temporary dependency failure (network blip, timeout) → Retry with backoff.
- A safe substitute or cached value exists → Fallback / Default.
- One bad record among many in a batch → Skip and Continue.
- System is in an unsafe or invalid state → Fail Fast.
- An optional feature is unavailable → Graceful Degradation.
- A dependency is repeatedly failing → Circuit Breaker.

KEY TAKEAWAY Ask two questions first: is the failure transient, and is there a safe fallback? The answers point straight at the right strategy.

TRY IT YOURSELF — EXERCISE 7: ERROR HANDLING STRATEGIES

Reinforces: the six error-handling strategies you just saw.

Goal

- Implement Retry and Fallback around a flaky call, then map the other four strategies to real scenarios

Retry

- fetchWithRetry(3) retries a transient failure up to 3 times before giving up

Fallback

- Returns a safe default (0) once retries are exhausted, instead of propagating

Map the rest

- One sentence each for Skip & Continue, Fail Fast, Graceful Degradation, Circuit Breaker

Key concept

- Pick the strategy based on whether the failure is transient and whether a safe default exists

Reference

- exercises/exercise-07-error-strategies.md

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-error-strategies.md.

LOGGING EXCEPTIONS FOR DIAGNOSTICS

Logging provides the information needed to diagnose issues, understand root causes, and resolve them faster.

WHY LOG EXCEPTIONS?

Quick Identification

Spot issues fast.

Context

When, where, why, how.

Root Cause Analysis

Enables real diagnosis.

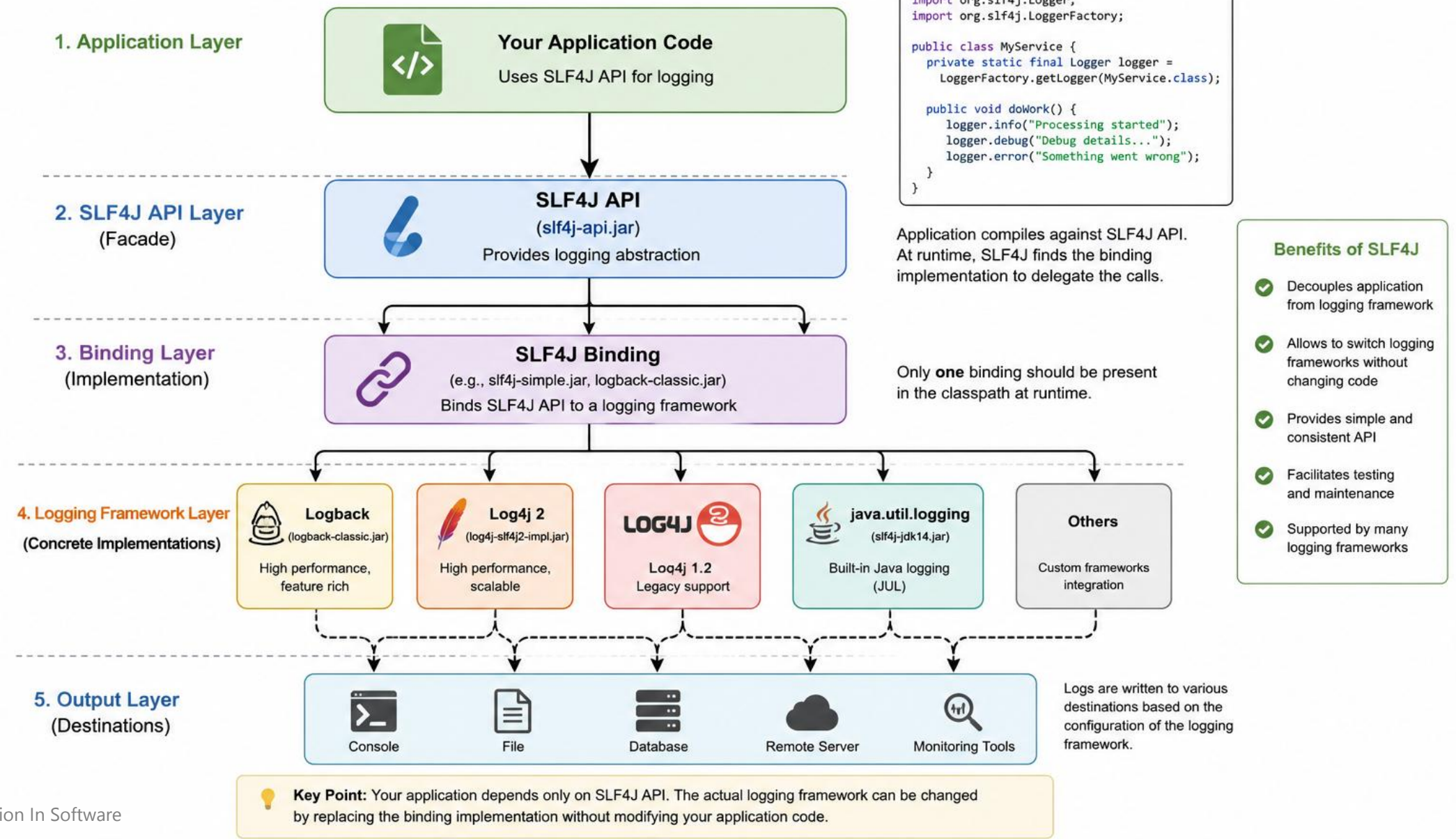
Monitoring & Alerting

Supports auditing too.

KEY TAKEAWAY A log entry without context — no request ID, no stack trace — is just noise, not a diagnosis.

SLF4J Architecture Overview

SLF4J (Simple Logging Facade for Java) is a logging facade that provides a simple API that delegates to various logging frameworks.




```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class MyService {
  private static final Logger logger =
    LoggerFactory.getLogger(MyService.class);

  public void doWork() {
     logger.info("Processing started");
     logger.debug("Debug details...");
     logger.error("Something went wrong");
  }
}
```

Application compiles against SLF4J API. At runtime, SLF4J finds the binding implementation to delegate the calls.

Only **one** binding should be present in the classpath at runtime.

Benefits of SLF4J

- Decouples application from logging framework
- Allows to switch logging frameworks without changing code
- Provides simple and consistent API
- Facilitates testing and maintenance
- Supported by many logging frameworks

Logs are written to various destinations based on the configuration of the logging framework.

Key Point: Your application depends only on SLF4J API. The actual logging framework can be changed by replacing the binding implementation without modifying your application code.

Logging Exceptions for Diagnostics in Java

Capture, log, store and analyze exceptions to quickly diagnose and fix issues.

1. Exception Occurs

An error occurs during program execution.

2. Catch Exception

```
try {
    riskyOperation();
} catch (Exception e) {
    // Pass exception
    // to logger
    logger.error("Operation
     failed", e);
}
```

Exception is caught and forwarded to the logger with context.

3. Log Exception

SLF4J + Logback

Logger records the exception details with timestamp, level, message and stack trace.

4. Store in Log

Logs are written to files / centralized logging system.

5. Analyze & Diagnose

Developers/DevOps analyze logs to identify root cause and take corrective action.

Example: Logging in Java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Service {
   private static final Logger logger =
       LoggerFactory.getLogger(Service.class);

   public void process() {
       try {
           riskyOperation();
       } catch (Exception e) {
           logger.error("Operation failed for userId={}",
                        userId, e);
       }
   }
}
```

Example: Log File Output

```
2025-05-20 10:15:30,123 ERROR [service-worker-1] com.example.Service -
Operation failed for userId=101
java.lang.NullPointerException: Cannot invoke "String.trim()"
because "input" is null
    at com.example.Service.process(Service.java:45)
    at com.example.Controller.handleRequest(Controllor.java:78)
    at com.example.App.main(App.java:15)

Caused by: java.lang.NullPointerException: input is null
    at com.example.Validator.validate(Validator.java:33)
    ... 2 more
```

Best Practices

- Log meaningful messages
- Always include stack trace (e)
- Avoid logging sensitive data
- Use appropriate log levels (ERROR)
- Centralize logs (ELK, Splunk, etc.)

Good logging turns exceptions into actionable insights and reduces mean time to resolution (MTTR).

WHAT TO LOG & EXAMPLE LOG ENTRY

Six data points, every time — plus what a well-formed log entry looks like.

- **Exception Type**
- **Error Message**
- **Stack Trace**
- **Timestamp**
- **Request/User Info**
- **App Context**

```
{
  "timestamp": "2026-07-12T10:32:01Z",
  "level": "ERROR",
  "exception": "NumberFormatException",
  "message": "For input string: \"abc\"",
  "requestId": "abc123", "userId": "8842"
}
```

KEY TAKEAWAY Use structured (JSON) logging so entries are easily searchable and correlatable across systems.

LOGGING EXAMPLE & LOG LEVELS

SLF4J + Logback — the standard Java logging combination — plus when to use each level.

```
private static final Logger log = LoggerFactory.getLogger(UserService.class);
try { parseAge(input); }
catch (NumberFormatException e) {
    log.error("Failed to parse age for userId={}", userId, e);
}
```

LEVEL	WHEN TO USE
ERROR	Something failed and needs attention now
WARN	Unexpected but recoverable situation
INFO	Normal but significant application events
DEBUG	Detailed diagnostic information for developers

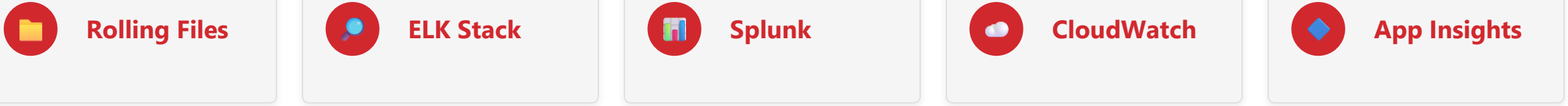
KEY TAKEAWAY Use parameterized logging (log.error("...{}", value)) to avoid string concatenation and improve performance.

GOOD LOGGING PRACTICES & DESTINATIONS

Where logs go, and the habits that keep them useful under pressure.

- Log exceptions with meaningful messages and always include the stack trace; avoid a bare printStackTrace() in production.
- Log the right context — correlation/request IDs, parameters, user, module; use appropriate log levels.
- Avoid logging sensitive data (passwords, tokens, PII); use structured (JSON) logging for better analysis.
- Keep log messages concise and consistent; ensure logs are easily searchable and correlatable.
- Example: ERROR [2026-07-12T10:32:01] userId=8842 requestId=abc123 NumberFormatException: "for input string: \"abc\""

LOG DESTINATIONS



KEY TAKEAWAY Logging flow: exception occurs → caught/handled → logged with context → stored → analyzed & alert triggered → issue resolved.

TRY IT YOURSELF — EXERCISE 8: CONTEXTUAL LOGGING WARM-UP

Reinforces: logging exceptions for diagnostics, as just covered.

Goal

- Log operational context and the stack trace while showing a safe user message

Context

- accountId = "A-1001" — a demo identifier, never a real PIN or secret

Log call

- LOGGER.log(Level.SEVERE, "Withdrawal failed accountId=" + accountId, ex)

User-safe message

- Show a short, generic message — never leak stack traces to the user

Key concept

- Good logs answer what happened, where, and why — without exposing secrets

Reference

- exercises/exercise-08-logging-warmup.md

TRY IT NOW Complete Exercise 8 before continuing — see exercises/exercise-08-logging-warmup.md.

SIX EXCEPTION-HANDLING MISTAKES

Poor exception handling can hide problems, complicate debugging, and lead to unreliable applications.

- Mistakes mask the real problem and increase downtime, make root cause analysis difficult, and can cause data corruption or security issues.

THE SIX MISTAKES

1. Catching Too Generically

- catch (Exception e) { } masks unexpected bugs.

2. Ignoring the Exception

- Empty catch hides issues; app continues inconsistently.

3. Not Logging

- Without logs, root cause is nearly impossible to find.

4. Only printStackTrace()

- Not user-friendly, hard to search, loses context in production.

5. Not Closing Resources

- FileInputStream left open → leaks, performance/stability issues.

6. Rethrow Without Context

- new RuntimeException(...) with no cause → loses the original exception.

KEY TAKEAWAY Each one destroys information the next developer will desperately need — never trade a stack trace for silence.

THE BETTER PATTERN & BEST PRACTICES

Catch specific, log with context, clean up, then recover or rethrow appropriately.

```
try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    return stmt.executeQuery();
} catch (SQLException e) {
    log.error("Query failed for sql={}", sql, e);
    throw new DataAccessException("Unable to fetch data", e);
}
```

- Catch specific exceptions, not generic ones; always log with meaningful messages and context.
- Close resources using try-with-resources; handle exceptions at the right level; rethrow with cause to preserve the stack trace.

KEY TAKEAWAY The Golden Rule: handle exceptions with care, provide context, and never ignore them — your future self will thank you!

ENTERPRISE ERROR MANAGEMENT BEST PRACTICES

Effective error management is critical for building resilient, secure, and user-friendly enterprise applications.

ERROR MANAGEMENT LIFECYCLE

WHY IT MATTERS

Business Continuity

Ensures reliability.

User Trust

Improves experience.

Lower MTTD/MTTR

Detect & resolve faster.

Security & Compliance

Enhanced posture.

Data-Driven

Continuous improvement.

KEY TAKEAWAY Great error management is proactive, consistent, and continuous — it reduces risk and drives better business outcomes.

ENTERPRISE ERROR MANAGEMENT BEST PRACTICES

JAVA

Build reliable, secure and maintainable enterprise applications

1. VALIDATE EARLY

Validate input and system state at the boundaries to prevent invalid data and fail fast.

2. USE APPROPRIATE EXCEPTIONS

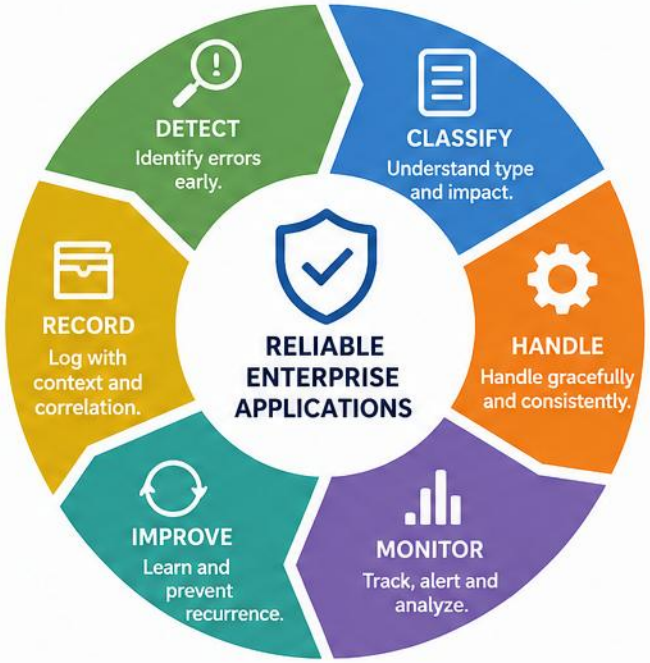
Throw specific exceptions that clearly describe the problem. Avoid generic Exception.

3. CONTEXT-RICH EXCEPTIONS

Include meaningful messages, error codes, and relevant context (IDs, parameters, user info).

4. CENTRALIZED HANDLING

Use global exception handling (e.g., @ControllerAdvice) to ensure consistent responses.



5. LOG EFFECTIVELY

Log errors with correlation ID, stack trace, user and request details. Use appropriate levels.

6. MONITOR & ALERT

Monitor error rates and critical failures. Set alerts and act proactively.

7. DON'T LEAK DETAILS

Do not expose internal error details to clients. Return safe, user-friendly messages.

8. TEST ERROR SCENARIOS

Test failure paths, edge cases and exception flows to ensure robust error handling.

FOUNDATIONAL PRINCIPLES

FAIL SAFE
Keep the system stable and protect data integrity.

USER CENTRIC
Provide clear, actionable messages for users and support teams.

SECURE BY DESIGN
Prevent information leakage and follow the principle of least privilege.

OBSERVABLE
Make errors observable with metrics, logs and dashboards.

CONTINUOUS IMPROVEMENT
Analyze root causes, improve processes and reduce error rates.

EXAMPLE FLOW

User Request → Validate → Error Occurs → Handle & Respond → Log & Correlate → Alert (If Critical) → Analyze & Improve

Java™

1. ROBUST ERROR HANDLING DESIGN & 2. CENTRALIZED LOGGING

Consistent patterns, applied across every layer of the application.

1. ROBUST ERROR HANDLING DESIGN

- Use consistent exception-handling patterns across every layer of the app.
- Handle errors at the appropriate layer; provide meaningful messages and recovery options.

2. CENTRALIZED LOGGING & MONITORING

KEY TAKEAWAY Log errors with context — timestamp, user, requestId, operation, environment — and monitor error rate, failure rate, latency, availability.

3. CONTEXT-RICH ERRORS & 4. SECURITY & COMPLIANCE

More context means faster root cause analysis — but never at the cost of exposing sensitive data.

3. CONTEXT-RICH ERROR INFORMATION: More context = faster root cause analysis.

4. SECURITY & COMPLIANCE

- Do not log sensitive data (PII, passwords, tokens, credit card numbers); mask or redact sensitive information.
- Enforce access control on logs and monitoring tools; retain logs based on compliance and data governance policies.
- Ensure audit trails for all critical errors.

KEY TAKEAWAY Security and diagnostics pull in opposite directions — context helps debugging, but never at the cost of leaking sensitive data.

5. RESILIENCE & AVAILABILITY & 6. CONTINUOUS IMPROVEMENT

Design for graceful degradation, then close the loop with a repeatable review cycle.

5. RESILIENCE & AVAILABILITY

- Use circuit breakers, retries with backoff, and fallbacks; design for graceful degradation.
- Handle transient failures differently from user or data errors; ensure idempotency for safe retries; test failure scenarios regularly.

6. CONTINUOUS IMPROVEMENT

KEY TAKEAWAY Close the loop: Learn → Improve → Prevent.

7. INCIDENT RESPONSE READINESS & CHECKLIST

The final piece: knowing exactly what to do when something goes wrong.

7. INCIDENT RESPONSE READINESS

- Have a documented incident response process; define roles and escalation paths.
- Use runbooks for common issues; communicate clearly with stakeholders; conduct post-incident reviews (PIRs) and blameless retrospectives.

BEST PRACTICES CHECKLIST

- Handle exceptions explicitly and consistently; log with rich context at the right level; monitor and alert proactively.
- Protect sensitive data and ensure compliance; design for resilience; review, learn, and improve continuously.
- Do not use exceptions for ordinary control flow; each catch block should represent genuine recovery, not routine logic.

KEY TAKEAWAY You can't prevent all errors, but you can manage them effectively — detect fast, respond smart, learn continuously.

LAB OVERVIEW — ATM BANKING SYSTEM

The official Lab 7 — build a fault-tolerant ATM console application under package com.academy.atm.

BUSINESS SCENARIO: a bank hardens its classroom ATM simulator — login with PIN retry limits, deposit, withdraw, transfer (with rollback), and mini statement — so invalid input never crashes the JVM and every error is logged.

- Prerequisites: Core Java (OOP, Collections, I/O), JDK 21, IntelliJ IDEA Community (or VS Code); completed pre-lab exercises 1–8 and Lab 0 setup.
- Skills practiced end-to-end: checked/unchecked exceptions, try-catch-finally, try-with-resources, throw/throws, custom exceptions, propagation, logging, and enterprise best practices — using plain console output (no external logging framework required).

KEY TAKEAWAY Every construct from this module — checked/unchecked exceptions, try-catch-finally, throw/throws, custom exceptions, propagation, and logging — comes together in one realistic ATM Banking System.

LAB TASKS AND DELIVERABLES

Core implementation steps, from project setup to a fully logged, menu-driven ATM console app.

1. Set Up Project — package com.academy.atm, logs/, and transactions.txt.
2. Create Custom Exceptions — InvalidAmountException, InsufficientFundsException, InvalidPinException, AccountNotFoundException.
3. Build Account & Transaction — throw on invalid amount/overdraft; track history.
4. Build LoggerUtil — timestamped INFO/ERROR entries to logs/application.log.
5. ATMService & Login — seed accounts; PIN retry limit (max 3) via multi-catch.
6. Deposit, Withdraw, Transfer — throw/catch/finally; rollback on failed transfer.
7. Mini Statement — try-with-resources reads transactions.txt.
8. Unchecked Exception Demos — handle NullPointer, Arithmetic, ArrayIndexOutOfBounds.
9. Menu-Driven Main — invalid choices recover; only option 7 exits.
10. Compile, Run & Verify — javac -d out; walk success/failure paths; inspect logs.

KEY TAKEAWAY This is the complete lifecycle of building an error-resilient ATM Banking System, from custom exceptions to verified logs.

LAB 7 DELIVERABLES & EVALUATION RUBRIC

Six deliverables to submit, graded against a 100-mark rubric.

DELIVERABLES

- Complete src/com/academy/atm project (Main, Account, Transaction, ATMService, LoggerUtil); four custom exception classes; transactions.txt.
- logs/application.log evidence; screenshots of success & failure paths; README (setup, hierarchy, logging strategy, lessons learned); reflection notes.

EVALUATION RUBRIC (100 MARKS)

- Project Structure 10 · Checked/Unchecked Handling 15 · Custom Exceptions 15 · try-catch-finally 15 · throw/throws 10 · Propagation 10 · try-with-resources 10 · Logging & Recovery 10 · Code Quality 5.

KEY TAKEAWAY Bonus stretch: transaction rollback, daily error report, PIN retry lock — plus the optional Week 1 Integrated Mini Project (Employee Management System v1).

MODULE 7 SUMMARY

You've learned how to effectively handle, propagate, and manage errors to build reliable, maintainable Java applications.

- Exceptions are events — they represent problems that occur during program execution.
- Handle close, throw far — handle exceptions where you can recover; propagate when you cannot.
- Clean up resources — always release resources using finally or try-with-resources.
- Informative errors — provide meaningful messages and log with context.
- Plan and standardize — use best practices and consistent strategies for enterprise reliability.
- Closing motto: handle with care, log with purpose, build with confidence.

EXCEPTION FLOW RECAP

KEY TAKEAWAY Good error management doesn't just handle failures — it prevents downtime, protects users, and builds trust.

MODULE 7 KNOWLEDGE CHECK — MULTIPLE CHOICE (1–5)

Choose the best answer. Correct answers are marked ✓.

1. What is the root of the Java exception hierarchy?

A) Exception B) RuntimeException **C) Throwable ✓** D) Error

2. What happens when an exception is not handled in the current method?

A) The program crashes immediately **B) It propagates up the call stack to the caller ✓** C) It is silently ignored D) It is auto-logged and discarded

3. Which mechanism automatically closes resources like files or connections?

A) finally block **B) try-with-resources ✓** C) throws clause D) Garbage Collector

4. Which strategy isolates a bad record and keeps the pipeline running?

A) Fail fast **B) Skip and continue ✓** C) Circuit breaker D) Ignore silently

5. What is the key difference between throw and throws?

A) They are identical keywords **B) throw raises an exception; throws declares it in a method signature ✓** C) throws raises an exception; throw declares it D) throw is only for checked exceptions

KEY TAKEAWAY Five more multiple-choice questions continue on the next slide.

MODULE 7 KNOWLEDGE CHECK — MULTIPLE CHOICE (6–10)

Choose the best answer. Correct answers are marked ✓.

6. How do you declare that a method may throw a checked exception?

A) Using throw in the method body **B) Using throws in the method signature ✓** C) Using catch in the method signature D) Checked exceptions cannot be declared

7. Which of the following is an unchecked exception?

A) IOException B) SQLException **C) NullPointerException ✓** D) FileNotFoundException

8. What is a best practice when creating custom exceptions?

A) Always extend Throwable directly **B) Provide meaningful messages and relevant context ✓** C) Never include a message D) Avoid custom fields entirely

9. What is a best practice for logging exceptions?

A) Only call e.printStackTrace() **B) Log with context, stack trace, and appropriate level ✓** C) Log passwords for debugging D) Avoid logging in production

10. Why do enterprise applications need good error handling?

A) It's optional for small teams **B) It reduces downtime, protects users, and builds trust ✓** C) It only matters for security teams D) It slows down development unnecessarily

KEY TAKEAWAY Core reminders: propagate when you cannot handle, handle when you can recover, and always log with stack trace and context.

WEEK 1 KEY TAKEAWAYS

Eight lessons that carry forward into every Java application you build from here.

1. Exceptions are Events — problems during execution that must be handled gracefully.
2. Use the Right Mechanism — try-catch, finally, or try-with-resources based on scenario.
3. Propagate When Needed — throw and throws to pass exceptions up the call stack.
4. Be Specific & Meaningful — custom exceptions and clear messages.
5. Log with Context — meaningful context at appropriate levels.
6. Handle Strategically — fail fast, retry, or graceful degradation.
7. Avoid Common Pitfalls — don't ignore, over-catch, or expose sensitive info.
8. Follow Best Practices — consistent management leads to trustworthy apps.

KEY TAKEAWAY These eight habits separate code that merely compiles from code that survives production — applications that fail safely instead of failing silently.

SKILLS GAINED & WHAT'S NEXT

You're ready to apply these concepts hands-on, in real-world scenarios.

SKILLS GAINED

Robust Handling

Design and implement it.

Custom Exceptions

Create and use them.

Propagation

Across layers, effectively.

Log & Analyze

For better diagnostics.

Enterprise Practices

Apply best practices.

WHAT'S NEXT?

Code → **Test** → **Analyze** → **Improve** → **Deliver**

Next, apply these concepts hands-on in Lab 7 — the ATM Banking System — to wrap up Week 1 before moving into Week 2.

KEY TAKEAWAY You've built the skills — Lab 7 is where they become muscle memory.

Week 1 complete

You can now handle, propagate, and manage errors to build reliable and resilient Java applications.

Hands-on labs and real-world projects are next in your toolkit.